HUMAN AUTHORITY & APPROVAL WORKFLOWS

An implementation-grounded architecture dossier | Written in my engineering voice | Aegis Agent

My architectural position. I designed Aegis Agent around a human-authorized local action lifecycle. In this document, I explain the specific controls visible in the public implementation, separate those controls from broader aspirations, and identify the boundaries I would require before making production-grade security claims. My evidence is source inspection; I have not executed the test suite as part of this publication.

01 | The implementation I can substantiate

In the cognitive-twin action path, I use a local SQLite-backed state machine with tenant checks, a SHA-256 digest of the canonical action envelope, an approval lifetime, a dry-run option, and local receipts or outbox artifacts. I also maintain separate risk-classification primitives. I do not present these components as proof of a fully integrated, independently authenticated two-person approval service.

Evidence level	What I can say
Source-inspected	The action functions, digest verification, TTL guard, tenant checks, API declarations and local artifact paths are present.
Test source inspected	Repository tests express expected behavior for approval, mutation denial and tenant separation.
Not established here	Passing test runs, production authentication, distributed concurrency guarantees and atomic cross-store commit.

02 | My separation of responsibilities

I distinguish policy classification from action-state enforcement. A policy label is not an authorization guarantee unless the execution path consumes and enforces it. The main persisted approval lifecycle is implemented in core/twin_actions.py; related but distinct policy models live in core/human_authority.py and core/twin_risk.py.

Implementation surface	Responsibility
core/human_authority.py	Heuristic score and AUTO / SINGLE_APPROVAL / DUAL_APPROVAL / DENY levels.
core/twin_risk.py	L0-L3 action classification and low-risk effect allow-list.
core/twin_actions.py	Propose, approve, reject, execute, digest, TTL, feedback and local effects.

Implementation surface	Responsibility
core/persistence.py	SQLite connection, WAL configuration and foreign-key settings.
core/twin_email_send.py	Approved email draft copied to local .eml outbox; no network delivery.
app/server.py	HTTP approval and cognitive-twin action route declarations.

03 | How I bind an approval to the exact action

I canonicalize an envelope containing action_id, tenant_id, kind, title, payload, effect_type, risk_level and policy_version. I hash the sorted, compact JSON representation with SHA-256. During approval, the supplied expected digest must match the current action. During execution, I recompute the digest and compare it with the stored approved digest.

This creates an integrity binding between approval and the represented action. It does not independently prove the identity of the approver, and it does not defend against an attacker who can rewrite both approval records and their supporting state.

04 | State transitions and authorization checks

Transition	Guard / observable outcome
Proposed → Approved	Action exists; tenant matches; status is proposed; expected SHA-256 matches. Actor, time, digest and feedback are persisted.
Proposed → Rejected	Reason and feedback are recorded. Tenant and digest validation depend on supplied arguments in this path.
Approved → Executed	Approval status, digest, TTL, training flag and effect restrictions are checked before local effect.
Approved → Dry run	Preflight applies; preview returned without status change or receipt write.
Executed → Executed	The already-executed action is returned idempotently.

I write the local artifact before marking the action executed. This means a failed artifact write can leave the action approved. The file write and SQLite status update are not one atomic transaction; I do not claim otherwise.

05 | My two distinct risk models

HumanAuthority score	Classification
Below 0.30	AUTO
0.30 to below 0.70	SINGLE_APPROVAL
0.70 to below 0.95	DUAL_APPROVAL
0.95 and above	DENY

I use deterministic signals including action length, selected high-risk words and a tenant-sensitive flag. Separately, my cognitive-twin risk classifier labels actions L0-L3 and constrains unknown L0 effects. These are heuristics and policy primitives, not a demonstrated semantic threat model or a universal enforcement fabric.

Importantly, DUAL_APPROVAL is a classification value; I have not established that two distinct authenticated humans must approve every action carrying that value. Likewise, AUTO in the classifier does not make the twin action lifecycle execute proposals automatically.

06 | Denial, expiry and recovery behavior

Condition	What the inspected path does
Unknown action or wrong tenant	Rejects the relevant operation; tenant enforcement varies by API/function arguments.
Execute before approval	Rejects with an approval-required error.
Envelope changes after approval	Digest mismatch prevents execution.
Approval exceeds TTL	Rejects execution; default lifetime is 24 hours.
Training-marked action	Execution is denied.
Unknown low-risk effect	Not silently accepted as a safe L0 effect.
Repeated execute	Returns prior executed result rather than writing a second receipt.

I use a process-local lock and SQLite WAL in the reviewed implementation. I do not describe them as proof of cross-process serializability or a distributed transaction protocol.

07 | Local effects, audit evidence and privacy boundary

I persist action metadata and approval/rejection feedback locally. Certain auxiliary audit or receipt writes are best-effort, so I do not claim every secondary audit sink commits successfully on every operation. For email actions, I write a tenant-scoped .eml draft labeled X-Aegis-Send: local-outbox-only; this is deliberately not an SMTP or network send. Other approved action paths write local receipts rather than executing arbitrary external commands.

08 | Engineering scenarios I use to explain the design

A. Changed proposal: If an action title or payload changes after the operator reviews its digest, the previously supplied digest no longer matches; the approval or execution path rejects it.

B. Missing approval: A proposed action cannot be executed as if it were approved. The implementation checks persisted state before local effects.

C. Dry-run preview: I can inspect an approved action through execute(..., dry_run=True) without marking it executed. I do not promise a complete email-body preview: the email branch returns empty preview bytes.

D. Local email draft: An approved email action creates a local outbox artifact; it does not send an external message. This distinction is part of my product boundary.

09 | My verification standard

Repository evidence	What I inspected
tests/test_human_authority.py	Low-risk classification, escalation and DENY behavior.
tests/test_t06_twin_actions.py	Consent, approval, rejection, pre-approval denial and API scenarios.
tests/test_t56_approval_binding.py	Digest binding, mutation rejection and approval metadata.
tests/test_approval_flows.py	Approval request/decision routes and evidence expectations.

I inspected these tests as source. I do not claim they passed in a current execution, and I do not present a previous CI outcome as a fresh verification of these paths.

10 | Claims I stand behind — and claims I withhold

I can substantiate: a tenant-aware, approval-bound local action workflow; SHA-256 integrity checks over a canonical action envelope; time-limited approvals; a dry-run path; local feedback; and risk-level policy primitives.

I do not claim: independently enforced two-person approval for all operations; arbitrary external action execution; network email delivery; guaranteed atomic audit persistence; or distributed, cross-process authorization guarantees.

11 | Public source evidence

My primary evidence is the public Aegis Agent repository: github.com/aminazimi42-coder/aegis-agent. Key files: core/twin_actions.py, core/human_authority.py, core/twin_risk.py, core/twin_email_send.py, core/persistence.py, core/session_receipt.py, app/server.py and the four test modules listed above. This document does not attribute features from unrelated private systems to this repository.

Authorship: Amin Azimi | AI Architect | Azimi Innovation Lab. The first-person voice represents my architectural explanation of the source-backed implementation, not an independent runtime certification.